

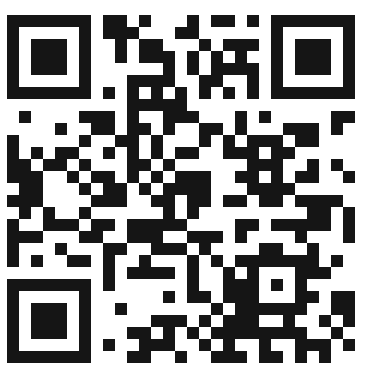
Succinct and Fast Tiny Pointer Hash Tables

Xilin Tang¹ Yuqi Mai¹ William Kuszmaul² Alex Conway³

¹Cornell University

²Carnegie Mellon University

³Cornell Tech

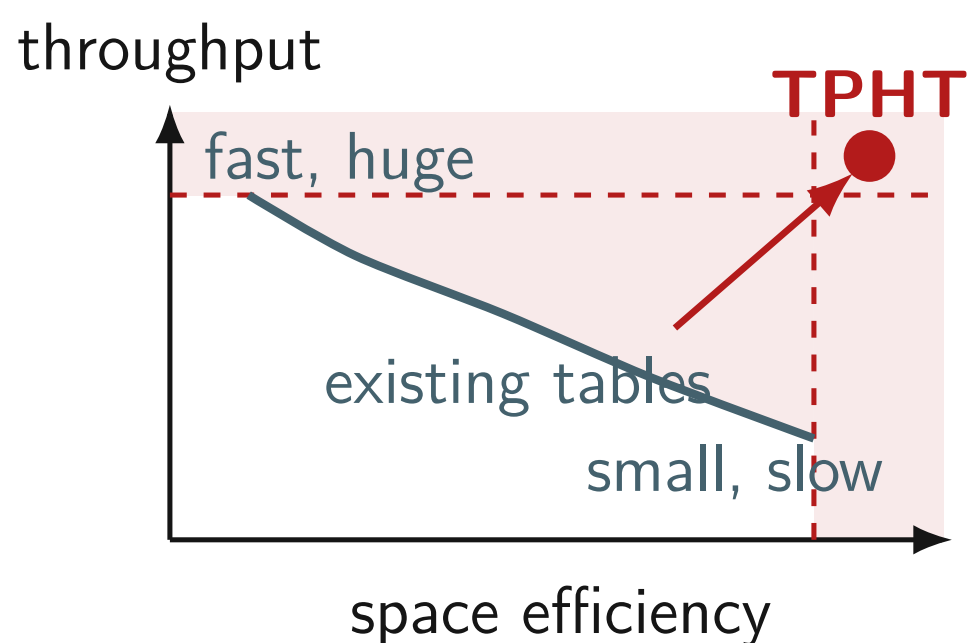


github.com/Xilinion/TPHT

A hash table can be **smaller than the data it stores** — and still **faster than the state of the art**.

What is a good hash table?

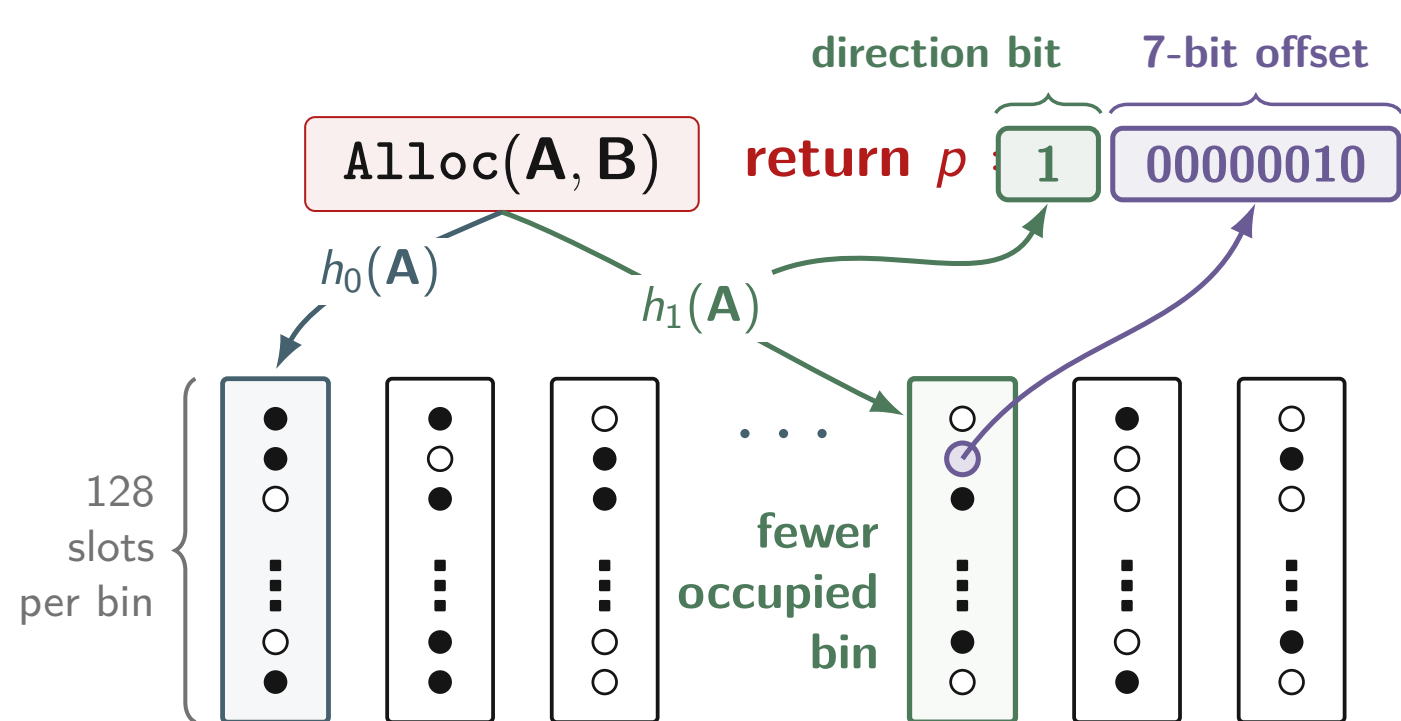
Every practical hash table trades throughput against memory: chaining pays for pointers, open addressing pays for empty slots. TPHT engineers two ideas from theory to escape the trade-off.



Two ingredients from theory

Ingredient 1: tiny pointers

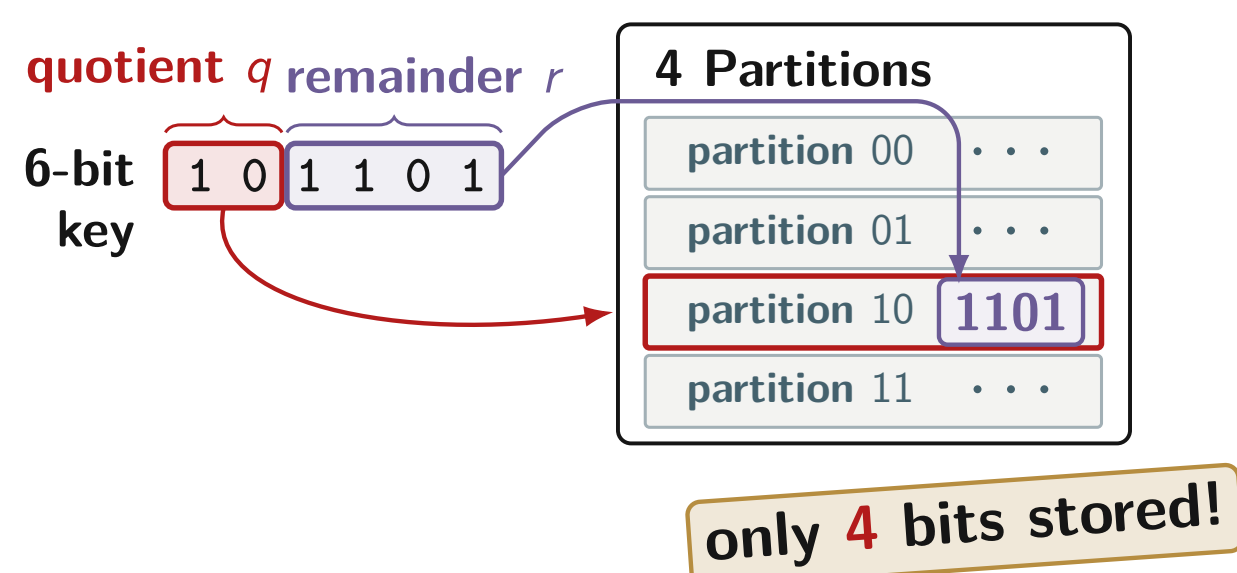
A **tiny pointer** can only be decoded together with its unique **owner** — the owner supplies the missing context, so the pointer shrinks to **one byte**: a direction bit plus an in-bin offset.



The backing allocator (a *dereference table*) hashes the owner to two candidate bins and picks the emptier one — and stays safe up to **95%** load.

Ingredient 2: quotienting

Split each key into quotient q and remainder r . The quotient **selects** the partition, so the location itself remembers q — only r is ever written down.

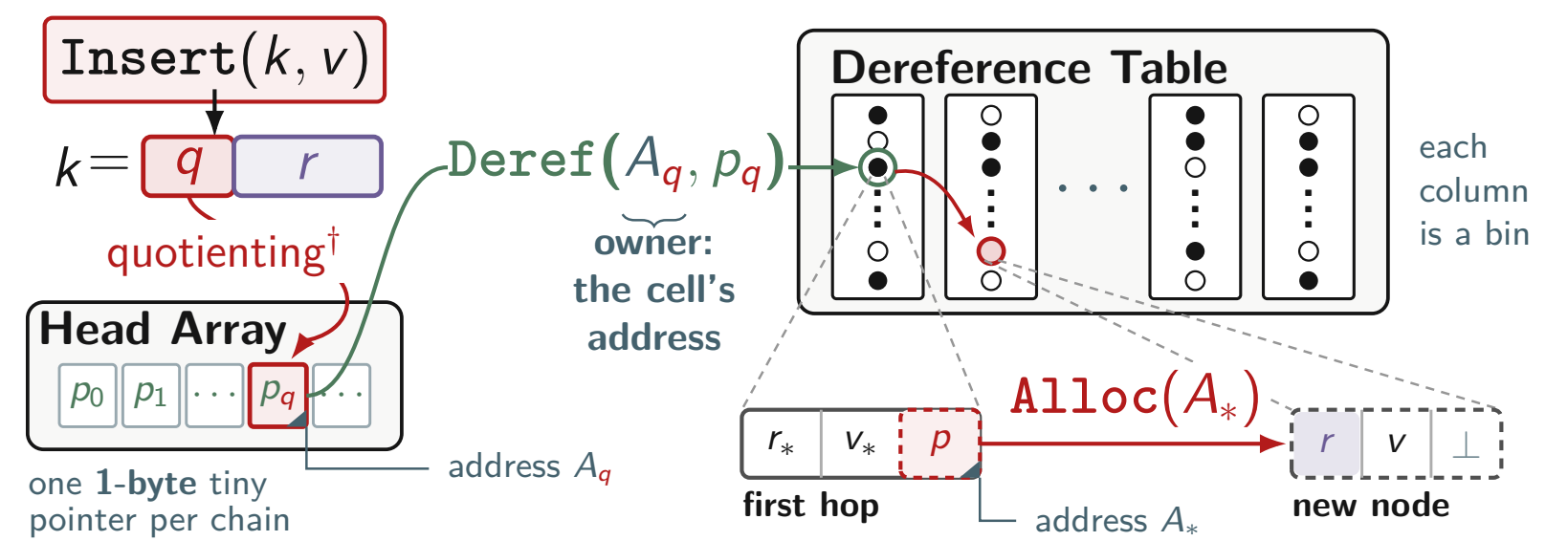


$k = q \circ r$ is always recovered exactly; a one-round Feistel permutation composes quotienting with hashing.

The TPHT designs

Chained-TPHT

To our knowledge the **first practical succinct hash table**: footprint below the raw data size, constant-time operations.

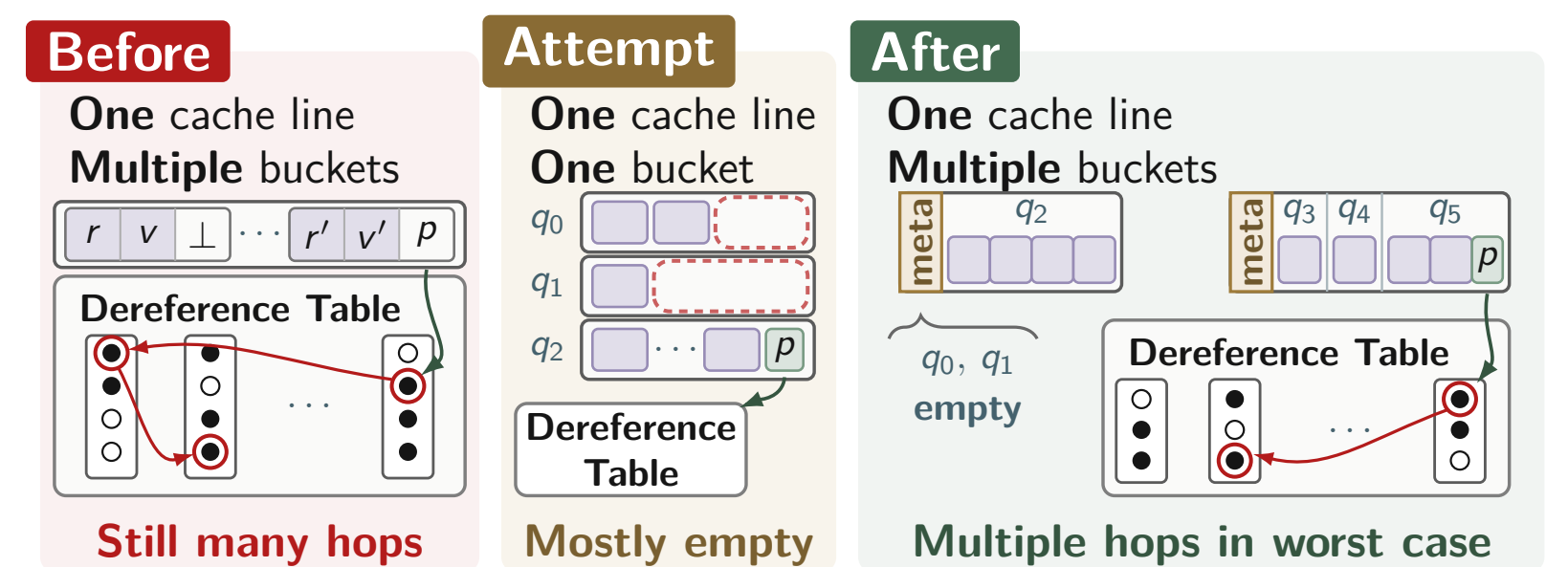


[†]The one-round Feistel permutation of Ingredient 2.

- Head array is **8×** smaller — heads stay in cache.
- The quotient is **never stored**; chains keep only (r, v, p) .
- First tuple inlined into the head entry: one fewer cache miss.

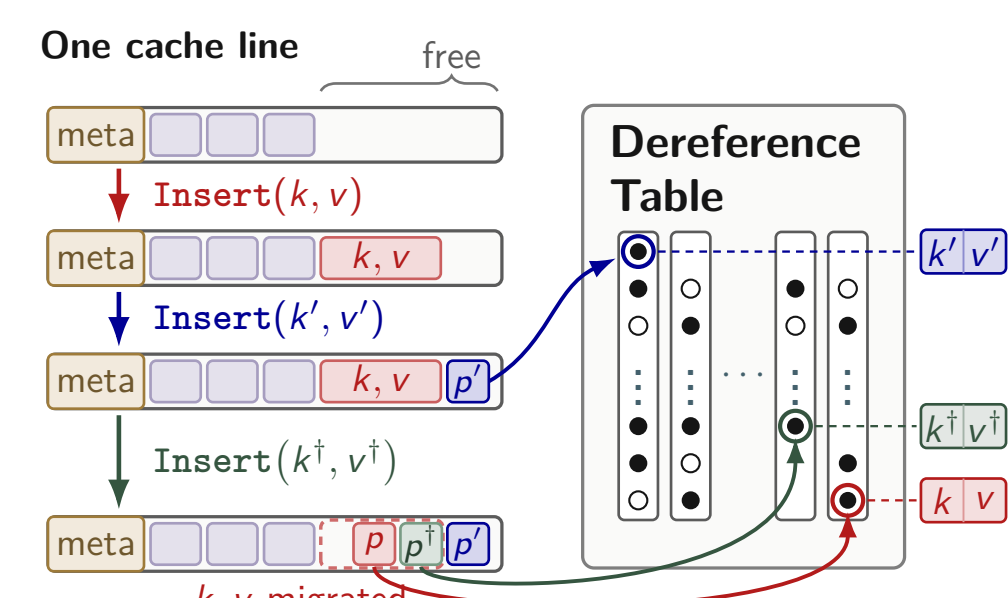
Making it fast: packing chains into cache lines

One cache line per chain? **Mostly empty**. Let **neighboring quotient groups** share one line: tight packing, but overflow still chains.



Flattened-TPHT: one-hop overflow

Keep the packing, drop the chains: a full home line migrates **one** tuple out, leaving a tiny pointer — overflow **never points to more overflow**.



- **1.25** memory accesses per operation — no pointer chasing, one hop, never a chain.
- Migration to the overflow table is rare: **5.1%** of tuples.
- Room for **31** tiny pointers per line; max load **24** w.h.p.

Built for real systems

- Online resizing — **no global pauses**, no stop-the-world rehash.
- Inserts, lookups, deletes, updates; 64-bit keys and values; multi-threaded.
- Library, benchmarks, and scripts open-sourced.

Evaluation

Workload: YCSB, 2^{26} =64M 64-bit key–value pairs, 16 threads. **Baselines:** libcuckoo^[1], IcebergHT^[2], Junction^[3], TBB^[4]. **Machine:** Intel Xeon Silver 4314, 16 cores, 24 MiB L3.

Headline numbers

105.4%

space efficiency:
Chained-TPHT's payload
exceeds its memory

2.3×

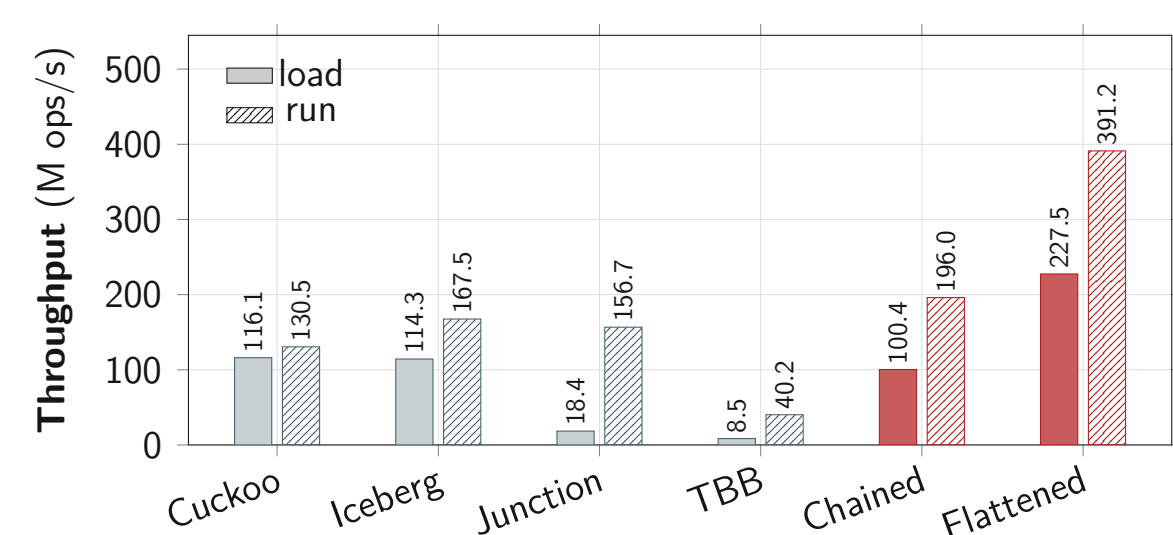
the fastest baseline's YCSB
throughput (Flattened-TPHT)

1.25

memory accesses per operation
— overflow never chains

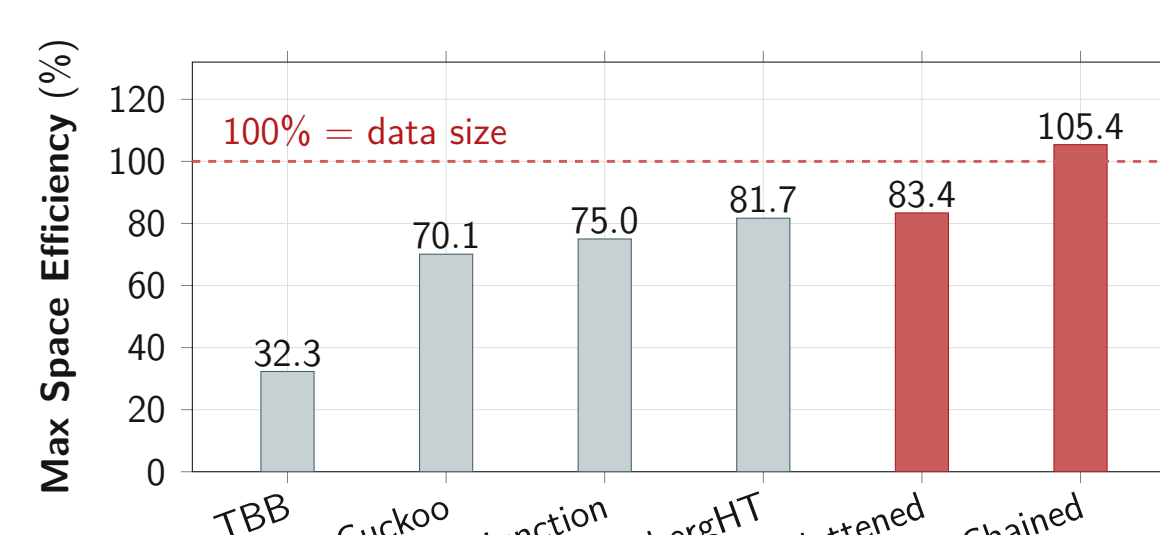
Result 1: higher throughput

Flattened-TPHT is **2.0×** the fastest baseline on load, **2.3×** on run.



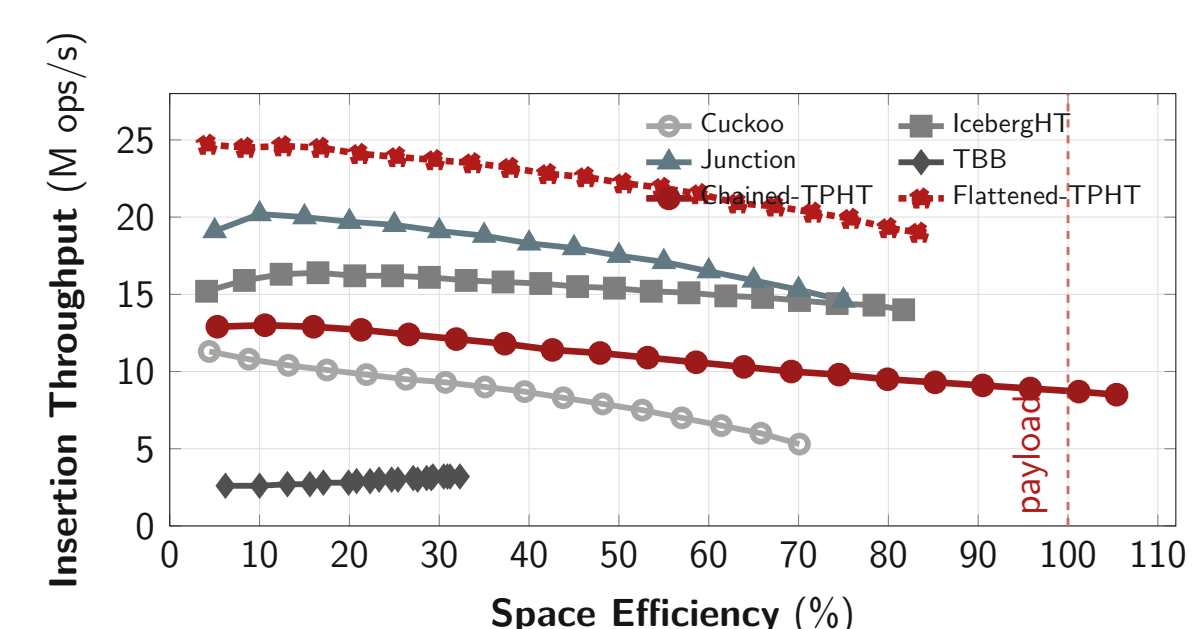
Result 2: space efficiency

Above **100%**, the table stores more payload bits than it allocates — quotienting's positional bits are free.



Result 3: speed vs. space

Single-core insertion sweep. A curve **ends where the table gives up**: baselines quit at 70–82%; Chained-TPHT runs past **105%**.



[1] Li, Andersen, Kaminsky, Freedman. *Algorithmic Improvements for Fast Concurrent Cuckoo Hashing*. EuroSys '14. [2] Pandey et al. *IcebergHT: High Performance Hash Tables Through Stability and Low Associativity*. SIGMOD '23. [3] Preshing. *Junction concurrent maps*. [4] Intel oneTBB concurrent_hash_map.